



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

Fundamentos Matemáticos para Programación Competitiva
Manual de Apoyo a la Docencia para ICPC

REPORTE DE ACTIVIDAD DOCENTE

QUE PARA OBTENER EL TÍTULO DE
Licenciado en Matemáticas

PRESENTA

Mariana Itzel Melo Tellez

TUTOR

Dr. Manuel Alcántara Juárez



Ciudad Universitaria, Ciudad de México, 2026.

Agradecimientos

Escribe aquí tus agradecimientos.

Índice general

1. Introducción	4
2. Introducción a la Programación Competitiva	5
2.1. ¿Por qué hacer programación competitiva?	7
2.1.1. Beneficios Académicos	7
2.1.2. Oportunidades laborales	8
2.2. Otros concursos y plataformas	8
2.2.1. Ejercicios	8
3. Entrada/Salida y Jueces en Línea	10
4. Teoría de números	11

Capítulo 1

Introducción

Este manual es una guía de apoyo para fortalecer la preparación matemática en programación competitiva, con enfoque en el Concurso Internacional Universitario de Programación (ICPC). Está dirigido tanto a estudiantes de matemáticas que buscan iniciarse en esta área como a participantes que desean consolidar y ampliar sus herramientas teóricas.

Si bien existen libros de programación competitiva ampliamente reconocidos, gran parte de ellos se concentra en algoritmos, implementación y estructuras de datos. Este manual busca complementar ese panorama con un enfoque explícito en la formación matemática que requiere la competencia y con una propuesta adaptada al contexto académico de la Facultad de Ciencias.

Además de la exposición conceptual, el manual incluye un conjunto de problemas cuidadosamente seleccionados y también problemas de autoría propia, diseñados para reforzar de manera directa los conocimientos presentados en cada tema.

La organización del contenido responde a áreas temáticas, no a niveles de dificultad, con el fin de que cada lector avance de acuerdo con sus intereses y necesidades. En el Capítulo 2 se presenta una introducción general a la competencia ICPC, su dinámica y sus reglas principales; en el Capítulo 3 se abordan los fundamentos de entrada/salida y el uso de jueces en línea; y en el Capítulo 4 se desarrolla el bloque de teoría de números aplicado a resolución de problemas.

Al final del documento se incluyen las editoriales de los problemas presentados. Estas editoriales comienzan con *hints* para orientar el proceso de resolución y no muestran de inmediato la solución completa. Para obtener mejores resultados, se sugiere seguir este orden de trabajo: primero intentar resolver cada problema por cuenta propia, después apoyarse en los *hints* cuando sea necesario y, finalmente, consultar la editorial para comparar enfoques y comprender la solución propuesta.

Capítulo 2

Introducción a la Programación Competitiva

La programación competitiva suele describirse como un deporte mental: exige constancia, disciplina y entrenamiento estructurado. El objetivo no es solamente encontrar una solución correcta, sino hacerlo con precisión y en el menor tiempo posible.

Su filosofía se centra en la resolución eficiente de problemas clásicos de ciencias de la computación. En este contexto, los enunciados están diseñados para ser resolubles con técnicas y algoritmos conocidos; es decir, no se trata de problemas de investigación abierta, sino de retos que evalúan la capacidad de modelar, implementar y optimizar soluciones bajo presión de tiempo. Resolver un problema implica comprender el enunciado, elegir una estrategia algorítmica adecuada y traducirla a código en un lenguaje de programación. El componente competitivo aparece cuando, además de resolver correctamente, se debe hacerlo rápido y con la menor cantidad de intentos fallidos.

En comparación con hackatones u otros concursos de desarrollo, donde se privilegia la construcción de productos completos durante varios días, la ICPC y competencias similares se enfocan en problemas puntuales con criterios estrictos de correctitud, tiempo y memoria. El sitio oficial de la ICPC la describe como una competencia algorítmica universitaria. En la práctica, esto significa que cada solución enviada es evaluada automáticamente por un programa llamado juez en línea que verifica si el programa responde correctamente a todos los casos de prueba y respeta los límites establecidos. Esta dinámica es una diferencia clave frente a concursos con evaluación más abierta y en ocasiones algo subjetiva, como los hackatones.

La ICPC es actualmente la competencia universitaria más representativa en esta disciplina. Se compete en equipos de tres estudiantes de la misma institución y, según la región, el proceso se organiza en varias fases clasificatorias. En el caso de México, desde 2024 se introdujo una estructura de **cuatro etapas**; entre ellas, el Gran Premio de México, compuesto por tres fechas de competencia clasificatorias. En cada fecha se resuelve un conjunto de problemas, usualmente entre 9 y 13, en tiempo limitado y la clasificación considera tanto la cantidad de problemas resueltos como la penalización acumulada por tiempo e intentos incorrectos.

Para hacerlo más claro e ilustrativo, tomemos como ejemplo el problema clásico de Codeforces

231A.

A. Team

time limit per test: 2 seconds

memory limit per test: 256 megabytes

One day three best friends Petya, Vasya and Tonya decided to form a team and take part in programming contests. Participants are usually offered several problems during programming contests. Long before the start the friends decided that they will implement a problem if at least two of them are sure about the solution. Otherwise, the friends won't write the problem's solution. This contest offers n problems to the participants. For each problem we know which friend is sure about the solution. Help the friends find the number of problems for which they will write a solution.

Input

The first input line contains a single integer n ($1 \leq n \leq 1000$), the number of problems in the contest. Then n lines contain three integers each, each integer is either 0 or 1. If the first number equals 1, then Petya is sure about the problem's solution; otherwise he isn't sure. The second number shows Vasya's view, and the third number shows Tonya's view.

Output

Print a single integer: the number of problems the friends will implement on the contest.

Example

Input

```
3
1 1 0
1 1 1
1 0 0
```

Output

```
2
```

Como se observa en este ejemplo, usualmente todo problema de programación competitiva consta de las siguientes secciones:

- **Tiempo límite:** tiempo máximo de ejecución permitido.
- **Memoria límite:** memoria RAM máxima para la solución.
- **Descripción:** contexto y tarea específica a resolver.
- **Input (entrada):** formato exacto de los datos de entrada.
- **Output (salida):** formato exacto de la respuesta esperada.
- **Ejemplos:** casos de prueba para validar la interpretación.

Una pregunta valiosa para quien inicia es: *¿entiendo lo que me pide el problema?*; y después, *ya que entendí el problema, ¿puedo crear un código para resolverlo?*; finalmente, *¿cuánto tiempo me tomaría escribir un código correcto y sin errores?*. Esta autoevaluación ayuda a medir el progreso real en

programación competitiva y, sobre todo, a construir confianza con cada intento. Incluso cuando la primera solución no sale a la perfección, cada ajuste mejora la lectura de problemas, la precisión del código y la velocidad de respuesta en concurso. Entre más problemas se resuelven, mayor “condición” se desarrolla para identificar patrones, tipos de enunciados, técnicas y trucos frecuentes; en consecuencia, cada vez resulta más natural acercarse a una solución correcta desde el primer intento.

La idea central para resolver el problema 231A es contar, en cada línea, cuántos valores son iguales a 1; si la suma es mayor o igual a 2, se incrementa el contador de problemas resueltos. Esta tarea permite practicar lectura de entrada, recorridos simples y condiciones lógicas. El código puede implementarse de forma directa:

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7     int implementados = 0;
8
9     for (int i = 0; i < n; i++) {
10         int a, b, c;
11         cin >> a >> b >> c;
12
13         // Si al menos dos están seguros, se implementa el problema.
14         if (a + b + c >= 2) {
15             implementados++;
16         }
17     }
18
19     cout << implementados << "\n";
20 }
```

En este ejercicio también queremos exponer un detalle importante: en la salida se imprime “\n”. Aunque parezca menor, el juez en línea suele comparar texto de forma estricta. Si el formato no coincide exactamente con lo esperado (por ejemplo, cuando el resultado debe ir en una línea y no se respeta), es posible recibir un veredicto **Wrong Answer** aun cuando la lógica sea correcta. Iniciar puede ser frustrante, pero después de superar esa curva de aprendizaje, este deporte se vuelve sumamente reconfortante.

2.1. ¿Por qué hacer programación competitiva?

2.1.1. Beneficios Académicos

La programación competitiva aporta beneficios académicos importantes para la formación en ciencias de la computación. En primer lugar, fortalece habilidades esenciales como el pensamiento algorítmico, el razonamiento lógico y la resolución eficiente de problemas. Al enfrentarse a ejercicios desafiantes, el estudiante aprende a descomponer tareas complejas en partes más pequeñas, detectar patrones y diseñar estrategias correctas y eficientes.

Además, funciona como un puente entre la teoría y la práctica. Conceptos vistos en clase –como estructuras de datos, análisis de complejidad, teoría de números o técnicas de optimización– dejan de ser abstractos cuando deben aplicarse para resolver problemas concretos bajo restricciones reales de tiempo y memoria.

Otro beneficio clave es que fomenta el aprendizaje continuo. Cada problema nuevo expone al estudiante a ideas distintas y lo motiva a investigar enfoques alternativos, comparar soluciones y refinar su forma de pensar. Con el tiempo, esto fortalece hábitos académicos valiosos: validar casos límite, justificar la correctitud de una solución y mejorar progresivamente la claridad del código.

2.1.2. Oportunidades laborales

La práctica constante en programación competitiva también tiene impacto directo en el perfil profesional. Muchas empresas tecnológicas evalúan a sus candidatos mediante entrevistas técnicas centradas en resolución algorítmica de problemas; por ello, entrenar este tipo de pensamiento representa una ventaja real en procesos de selección para pasantías y puestos de tiempo completo.

Durante estas evaluaciones no solo importa llegar a la respuesta correcta. También se observa cómo razona la persona candidata, cómo comunica sus ideas, cómo depura errores, cómo justifica la complejidad de su solución y cómo se adapta cuando cambian las condiciones del problema. Todas estas competencias se trabajan de forma natural en competencias de programación.

Finalmente, la experiencia en concursos también puede aportar visibilidad profesional: buenos resultados, constancia y participación activa en comunidades técnicas suelen reflejar iniciativa, disciplina y capacidad de trabajo intelectual bajo presión, cualidades altamente valoradas en la industria.

2.2. Otros concursos y plataformas

Además de la ICPC, existen competencias en las que puede participar cualquier persona, sin necesidad de ser estudiante. Algunos ejemplos son Google Hash Code, Google Kick Start y Mexico ICPC Masters.

También hay plataformas que organizan concursos de práctica de forma regular, como Codeforces y AtCoder. Estos espacios son ideales para entrenar de manera constante, comparar enfoques con otras personas y acostumbrarse a resolver problemas bajo presión de tiempo.

En resumen, existen muchos jueces en línea y miles de problemas disponibles para seguir practicando. Siempre habrá un nuevo reto esperándote.

2.2.1. Ejercicios

- Tabla para Pastel (omegaUp): Concurso FCiencias 2016.

- Watermelon (Codeforces 4A): Determinar si una sandía de peso w puede dividirse en dos partes pares positivas.
- Minimize (Codeforces 2009A): Para cada caso, hallar el mínimo de $(c - a) + (b - c)$ con $a \leq c \leq b$.
- I Wanna Be the Guy (Codeforces 469A): Decidir si dos jugadores, combinando los niveles que cada uno puede pasar, cubren todos los niveles del juego.

Capítulo 3

Entrada/Salida y Jueces en Línea

En el capítulo anterior mencionamos los jueces en línea (online judges), a grandes razgos, estos son sistemas que reciben un programa enviado por un usuario, entonces ejecutan este con entradas predefinidas a las que se les llama casos y compara las respuestas arrojadas por el programa con las esperadas, y a partir de esto determina si la solución enviada es correcta o no. El funcionamiento de un juez en línea se puede ver de la siguiente manera:

Cliente → Frontend → Backed → Evaluador

Capítulo 4

Teoría de números

En programación competitiva, el dominar o al menos tener buenas nociones de los conceptos principales de teoría de números no es un lujo, es una necesidad, el lector se dará cuenta a lo largo de este texto y en su aventura resolviendo problemas en la ICPC que muchos de los algoritmos usan alguna noción o idea de teoría de números.

Algoritmos básicos

Dados dos números a y b , nuestra tarea será encontrar el mayor número que es divisor tanto de a como de b , por ejemplo, si $a = 6$ y $b = 4$ tendríamos que el mcd (maximo común divisor) sería 2. Ahora ya que no es práctico intentar todos los divisores, presentaremos un algoritmo llamado **algoritmo de euclides** que es más práctico para encontrar este número. Este algoritmo se centra en la afirmación de que dados dos números a y b , de forma de b no divida a a , si escribimos $a = bq + r$ entonces $\text{mcd}(a, b) = \text{mcd}(b, r)$, de aquí que podamos escribir lo siguiente:

$$a = bq + r_1$$

$$b = r_1q_1 + r_2$$

$$\vdots$$

$$r_{n-1} = r_nq_n$$

y por la afirmación anterior, $\text{mcd}(a, b) = \text{mcd}(b, r_1) = \text{mcd}(r_1, r_2) = \dots = r_n$, nótese que además $r_1 = a \bmod b$, de forma que algorítmicamente tendríamos que el $\text{mcd}(a, b) = a$ si $b = 0$, de lo contrario $\text{mcd}(a, b) = \text{mcd}(b, a \bmod b)$ y así podemos escribir el siguiente código:

```
1 int mcd(int a, int b) {  
2     if (b==0)  
3         return a;  
4     else  
5         return mcd(b, a % b );  
6 }
```

Esta función no es necesario implementarla pues está incluido como función estándar desde C++ 17 con el nombre de *gcd* (greatest common divisor).

Además de obtener este código, este algoritmo nos dice que $\text{mcd}(a, b) = ax + by$ para algunos enteros x, y , es decir, $\text{mcd}(a, b)$ se puede ver como combinación lineal de a y b , esto se le conoce como el lema de Bezout.

Otra tarea que nos podría parecer interesante es dados dos números a y b , cómo calcular el mínimo común múltiplo (denotado mcm), esto es, el número menor que es divisible por tanto a como b . Este problema en realidad está casi resuelto con nuestro código de arriba, esto es por una propiedad importante del mcd :

$$\text{mcm} = \frac{ab}{\text{mcd}(a, b)}$$

de forma que nuestra implementación sería simplemente calcular la fórmula de arriba, sin embargo aquí un detalle importante a considerar es que si nuestros enteros son muy grandes, puede que tengamos un overflow al multiplicar ab , de forma que podemos primero dividir y después multiplicar, tomar en cuenta que decisiones de esta índole aunque simples pueden evitarnos veredictos incorrectos en muchos problemas:

```
1 int mcm(int a, int b){  
2     return (a/mcd(a,b))*b;  
3 }
```

Aritmética modular

Iniciemos analizando un ejemplo.

Ejemplos [3.1] Supongamos que hoy es Lunes, ¿qué día de la semana será dentro de 300 días?

Solución. Podríamos hacer una lista de 300 números después del lunes, asignando cada día a cada número y ver cuál queda en el número 300, pero esto sería muy tardado y realmente innecesario, pues si notamos que cada 7 días se repite el mismo día, podemos ver que ya que al dividir 300 entre 7 nos da 42 y nos queda de residuo 6, por lo que sabemos que el día 264, es decir 42 veces 7, será Lunes y sólo tenemos que ver los restantes 6 días, que sabiendo que 6 días después de Lunes es Domingo, tendremos nuestra respuesta.

En este ejemplo vemos como la noción de que conjuntos de números se conformen de forma cíclica nos simplifica problemas. Si en el ejemplo anterior en lugar 300 días fueran 314 días, la respuesta sería la misma, podría parecer una afirmación fuerte, pero realmente al dividir 314 entre 7 como hicimos en el ejemplo notaremos que el residuo es el mismo, es decir 6, entonces el día 308, es decir 44 veces 7, será Lunes y tenemos nuevamente que ver los siguientes 6 días, esto nos lleva a pensar que si de días de la semana estamos hablando, dos días representan el mismo día de la semana si y sólo si tienen el mismo residuo al dividirlos entre 7.

Resulta que esto mismo lo podemos hacer con cualquier número natural n , sea n un número natural. Si x y y son dos números enteros tales que tienen mismo residuo al dividirlos entre n diremos que x es congruente a y módulo n lo que podemos escribir con la siguiente notación:

$$x \equiv y \pmod{n}$$

Antes de ver ejemplos, veamos algunas propiedades de las congruencias, si $a \equiv b \pmod{n}$ y $c \equiv d \pmod{n}$ y m es un entero positivo, entonces lo siguiente se cumple:

$$a + c \equiv b + d \pmod{n}$$

$$ac \equiv bd \pmod{n}$$

$$a^m \equiv b^m \pmod n$$

Como podemos notar, las congruencias respetan las sumas, productos y potencias como estamos acostumbrados, y en realidad para sumas y productos ocurre que cualquier número puede sustituirse por otro al que sea congruente en el módulo de la congruencia.

Inverso Multiplicativo Modular

Aunque pueda parecer que las operaciones con congruencias son muy flexibles, hay que tener cuidado, pues aún no hemos mencionado nada sobre la división. Suena natural querer definir un número que sea un inverso multiplicativo, es decir dado un número a y un número n , encontrar un número b tal que $ab \equiv 1 \pmod n$, a este lo denotaremos como a^{-1} .

Es importante mencionar que este inverso no siempre existe. Tomemos por ejemplo el caso de $n = 4$ y $a = 2$, es fácil ver que revisando todas las posibilidades de b módulo n no existe un número que cumpla lo pedido. Se puede probar que este inverso existe si y solo si $\text{mcd}(a, n) = 1$.

Ahora, también podemos diseñar un algoritmo para encontrarle en caso de que exista, recordemos que arriba en esta sección vimos el algoritmo de euclides para poder encontrar el mcd de dos números, y en particular teníamos que $\text{mcd}(a, b) = \text{mcd}(b, r_1) = \dots = \text{mcd}(r_{n-1}, r_n) = \text{mcd}(r_n, 0) = r_n$, donde los r_i eran los residuos, también por el lema de Bezout tenemos que existen x y y tal que $\text{mcd}(a, b) = ax + by$, pues bien, queremos encontrar esos coeficientes y notemos que tenemos ya una combinación lineal trivial con r_n y 0 de la que conocemos los coeficientes, pues de lo anterior

$$\text{mcd}(a, b) = \text{mcd}(r_n, 0) = 1r_n + 0$$

entonces intentaremos usar ingeniería inversa de las ecuaciones del algoritmo de euclides para encontrar estos x y y para a y b iniciales, es decir si antes teníamos que en cada caso a y b pasaban a ser b y $a \pmod b$, ahora queremos ver conociendo x_i y y_i en una ecuación $x_i b + y_i a \pmod b = \text{mcd}(a, b)$ quiénes son x y y en la ecuación $xa + yb = \text{mcd}(a, b)$, nótese que ya que $a \pmod b = a - (\lfloor \frac{a}{b} \rfloor * b)$ podemos sustituir y tenemos que

$$\text{mcd}(a, b) = x_i b + y_i (a - (\lfloor \frac{a}{b} \rfloor * b)) = ay_i + b(x_i - \lfloor \frac{a}{b} \rfloor y_i)$$

Sin embargo, aunque el algoritmo anterior funciona, para algunos problemas será importante que podamos calcular este inverso en tiempo lineal, entonces presentaremos un método usando exponenciación binaria

Métodos de factorización

Ejercicios

[3.1] Definamos $S(x)$ como la suma de dígitos de un número x en sistema decimal. Por ejemplo $S(11) = 2$, $S(1002) = 3$, $S(5) = 5$.

Diremos que un entero x es interesante si $S(X + 1) \leq S(X)$. En cada test se te dará un número n . Tu tarea es calcular cuántos números x cumplen que $1 \leq x \leq n$ y x es interesante.

Input:

La primera línea contiene un entero t ($1 \leq t \leq 1000$) — número de casos.

Después le siguen t líneas, la i -ésima línea contiene un número entero n ($1 \leq n \leq 10^9$).

Output:

Imprima t enteros, el i -ésimo entero deberá ser la respuesta para el i -ésimo caso.

Ejemplos:

Entrada	Salida
5	0
1	1
9	1
10	3
34	88005553
880055535	

[3.2] Un número $1 \leq x \leq 10^9$ es elegido. Te darán dos enteros a y b , los dos divisores más grandes del número x y también se cumple que $1 \leq a < b < x$.

Para los números dados a, b tienes que encontrar el valor de x .

Input:

Cada test contiene varios casos. La primera línea contiene un entero t ($1 \leq t \leq 10^4$), el número de casos. Luego la descripción de los casos.

La única línea de cada caso contiene dos enteros a, b ($1 \leq a < b \leq 10^9$).

Se garantiza que a, b son los divisores más grandes para algún número $1 \leq x \leq 10^9$.

Output:

Para cada caso de prueba, imprime el número x , tal que a y b son los mayores divisores del número x . Si hay varias respuestas, imprime cualquiera de ellas.

Ejemplos:

Entrada	Salida
8	6
2 3	4
1 2	33
3 11	25
1 5	20
5 10	12
4 6	27
3 9	1000000000
250000000 500000000	